Simulating a Pneumatic Soft Finger in MATLAB

A step-by-step record of every simulation, what it does, and what it showed

Author: Hiba Khan **Prepared with:** Claude (connected to MATLAB R2025b)

Project folder: C:\Users\HIBA KHAN\OneDrive\Documents\MATLAB SOFT FINGER

October 2026

In one paragraph: We modelled a single air-powered rubber finger (the kind used in soft grippers) entirely in MATLAB, without CAD. We started from how the rubber stretches, worked out how far the finger bends at a given air pressure, tested which design choices matter, simulated how fast the air fills and empties the finger, built a controller that holds the finger at a chosen angle, estimated how hard it can squeeze, and finally rebuilt the finger in SoRoSim, a research-grade soft-robot simulator, to cross-check our simpler model. This document explains each step in plain words.

Contents

1. Why we did this, and the finger we chose
2. Setting up: connecting Claude to MATLAB, toolboxes, SoRoSim
3. Step 1 – How the rubber stretches (material model)
4. Step 2 – How far it bends for a given pressure
5. Step 3 – Which design choices matter (parameter sweep)
6. Step 4 – How fast air fills and empties it (dynamics)
7. Step 5 – Holding an exact angle (cascade PID control)
8. Step 6 – How hard it can squeeze (grip force)
9. Step 7 – Getting SoRoSim working (built-in demo)
10. Step 7b – Building our finger in SoRoSim and comparing
11. Problems we hit and how we fixed them
12. What the model can't tell us yet, and the measurement plan
13. File list

1. Why we did this, and the finger we chose

Soft grippers, like the ones in Prof. Deepak Kumar's lab at MANIT Bhopal, grip by pumping air into rubber chambers. The hard part of these robots is not the electronics; it is predicting and controlling how a squishy, stretchy body moves. Before getting into the lab, we wanted to understand that physics with simulations alone, using only MATLAB at home (no CAD software).

We picked one simple, common design, the **PneuNet-style finger**: a rubber bar with an air chamber running along it, sitting on a bottom layer that is hard to stretch. When air pushes in, the top stretches but the bottom can't, so the finger curls toward the bottom, just like a real finger bending.

The baseline finger

Property	Value	Plain meaning
Length L	100 mm	Tip to base
Width b	20 mm	Side to side
Height H	17 mm	Bottom to top
Air chamber	from 3 mm to 15 mm height, 2 mm side walls	Hollow channel the air fills
Chamber area A_c	192 mm²	Area the air pushes on (cross-section)
Chamber centre y_c	9 mm above the bottom	Lever arm of the air force
Material	Ecoflex 00-30 silicone	Very soft, common lab rubber
Shear modulus μ	25.4 kPa (C_{10} = 12.7 kPa)	How stiff the rubber is

Why the bottom layer matters: the bending happens because the top and bottom stretch by different amounts. If both stretched equally, the finger would just get longer, like a balloon.

2. Setting up: connecting Claude to MATLAB

2.1 The MATLAB connection

Claude can't open MATLAB by itself. We installed MathWorks' official **MATLAB MCP server** as an extension in the Claude desktop app. It lets Claude write code, run it in MATLAB on your computer, and read back the results and plots. We also installed the **Filesystem** extension so Claude could read and write files in the project folder.

Getting it to connect took a few tries:

- **Error 1:** "Failed to create file C:\Program Files\MATLAB\R2025b\.matlab-mcp-server". The MATLAB install path had been typed into the *Log Folder* box by mistake, and Windows doesn't let programs write inside Program Files.
- **Error 2:** "Failed to initialize MCP Server". This is a known problem with the default temporary log folder.
- **Fix:** set *MATLAB Installation Path* = C:\Program Files\MATLAB\R2025b, set *Log Folder* to the project's own `logs` folder, and set session mode to *new*. It then connected.
- **Path tip:** the folder name has a space in it ("HIBA KHAN"), so paths need quotes, or Windows splits them.

2.2 Toolboxes and SoRoSim

Steps 1–6 use only core MATLAB. Step 7 uses **SoRoSim**, a free soft-robot simulator from Khalifa University based on Cosserat rod theory (it treats the finger as a flexible rod that can bend, twist and

stretch). SoRoSim needs two extra toolboxes, which you installed:

- Optimization Toolbox (to solve for the shape where all forces balance)
- Symbolic Math Toolbox (used inside SoRoSim's setup)
- SoRoSim itself, installed as a MATLAB add-on (the official Differentiable_SoRoSim release)

2.3 Two compatibility patches ("shims")

SoRoSim shipped with two problems on our MATLAB version, so we added small patch files in a folder called `sorosim_shims`, placed ahead of SoRoSim on MATLAB's path:

- **MEX mismatch:** SoRoSim ships pre-compiled fast versions (MEX files) of its maths functions, built with a different MATLAB Coder version, so they refused to run. Each shim (e.g. `SoftJointKinematics_Z2_mex.m`) has the same name as the broken file but simply calls the normal MATLAB source code instead. That's slower, but it works. The rigid-joint ones also reshape empty inputs to the size MATLAB expects.
- **Cable GUI bug:** the window for adding a cable saved an empty field (`div_endi`), which crashed `CableActuation` at line 33. Our replacement `CableActuation.m` skips that window and creates the one cable we need directly: a straight cable at y = 0, z = –8 mm running the whole finger. It prints *"Soft finger shim: 1 cable at y = 0 m, z = -0.008 m along the full soft link."*

3. Step 1 – How the rubber stretches

File: step1_material_model.m

What we wanted to know

Rubber doesn't behave like a spring. Stretch it a little and it's easy; stretch it a lot and it gets much stiffer. Before anything else we needed a formula linking *how much the rubber is stretched* to *how hard it pulls back*. Engineers call these **hyperelastic models**.

What we did

We used the **stretch ratio** λ (new length ÷ original length; $\lambda = 1.5$ means 50% longer) and compared two standard models:

- **Neo-Hookean**, the simplest rubber model with one stiffness number: stress $P = 2(\lambda - \lambda^{-2})\cdot C_{10}$.
- **Yeoh**, which adds extra terms so the rubber stiffens more at large stretch, as real silicone does.

We took real material numbers from published tests (Runge et al., arXiv 2209.03831), not guesses:

Material	C_{10} (kPa)	Shear modulus $\mu = 2C_{10}$ (kPa)
Ecoflex 00-30	12.7	25.4
Ecoflex 00-50	25	50
Dragon Skin 10	42.5	85
Elastosil M4601 (Yeoh C_{20} = 20 kPa)	131.2	262.4

What we found

Stretch λ	1.1	1.3	1.5	2.0	2.5
Yeoh above neo-Hookean by	+1%	+8.3%	+21.2%	+72.7%	+147%

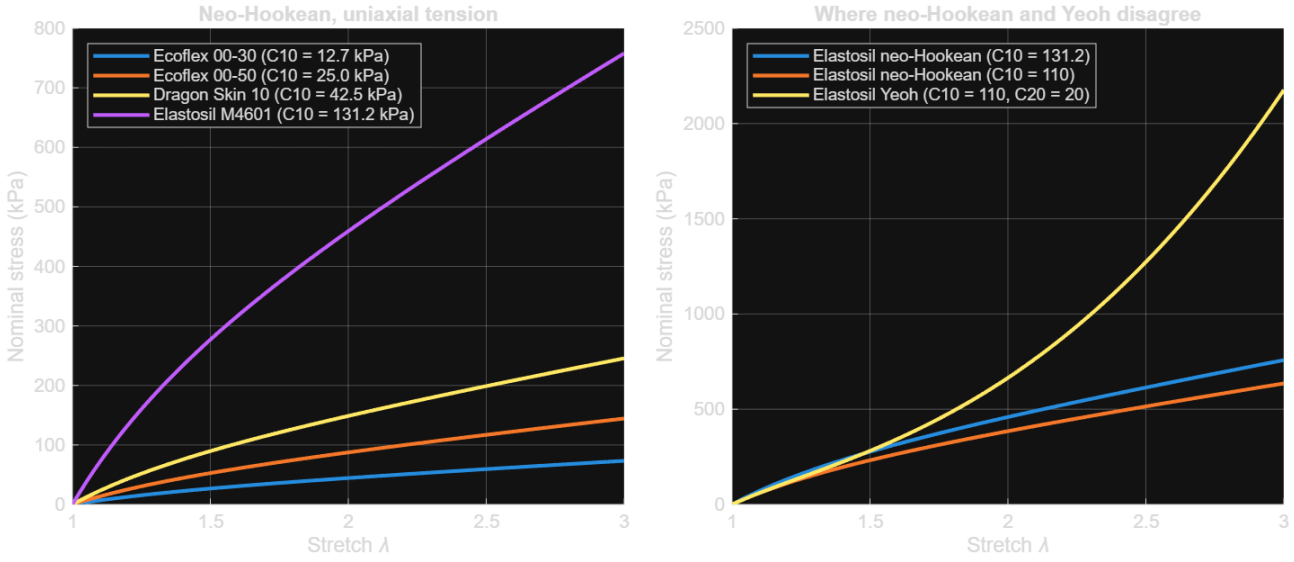


Figure 1. Stress vs stretch for the neo-Hookean and Yeoh models, and the four materials.

What it means: up to about 30% stretch the simple neo-Hookean model is good enough (under 10% error). Our finger mostly works in that range, so we used neo-Hookean for the bending model. Remember that above roughly 50% stretch it underestimates stiffness, so it will over-predict bending at high pressure.

4. Step 2 – How far it bends for a given pressure

File: step2_pressure_bend.m **Plot:** step2_bend_vs_pressure.png

The idea (a see-saw of turning forces)

Air pressure P pushing on the chamber area A_c makes a force. Because the chamber sits y_c = 9 mm above the stiff bottom, that force creates a turning effect (a **bending moment**): $M_{air} = P \cdot A_c \cdot y_c$.

As the finger curves (curvature κ, i.e. 1 ÷ bend radius), every slice of rubber at height y gets stretched to $\lambda = 1 + \kappa y$. Higher slices stretch more. Each slice pulls back with the Step 1 stress, and adding up all those pulls times their height gives the rubber's resisting moment $M_{rubber}(\kappa)$.

The finger settles at the curvature where the two balance: $\mathbf{P \cdot A_c \cdot y_c = M_{rubber}(\kappa)}$. MATLAB's `fzero` finds that κ for each pressure. The total bend angle is then $\theta = \kappa \times L$.

One detail: at chamber height (3–15 mm) only the two 2 mm side walls carry stress, because the middle is air. Above and below the chamber the full 20 mm width does. The function `bw(y)` handles this.

```
s   = @(lam) mu*(lam - 1./lam.^2);                    % neo-Hookean stress
bw  = @(y) b*(y<yb | y>yt) + 2*wall*(y>=yb & y<=yt);  % solid width at height y
Mres = @(k) integral(@(y) s(1+k*y).*y.*bw(y), 0, H, 'Waypoints',[yb yt]);
k   = fzero(@(k) P(i)*Ac*yc - Mres(k), [0 500]);       % balance the moments
theta = rad2deg(k*L);                                 % bend angle in degrees
```

What we found

Pressure	10 kPa	20 kPa	25 kPa
Bend angle	110.9°	271°	368°

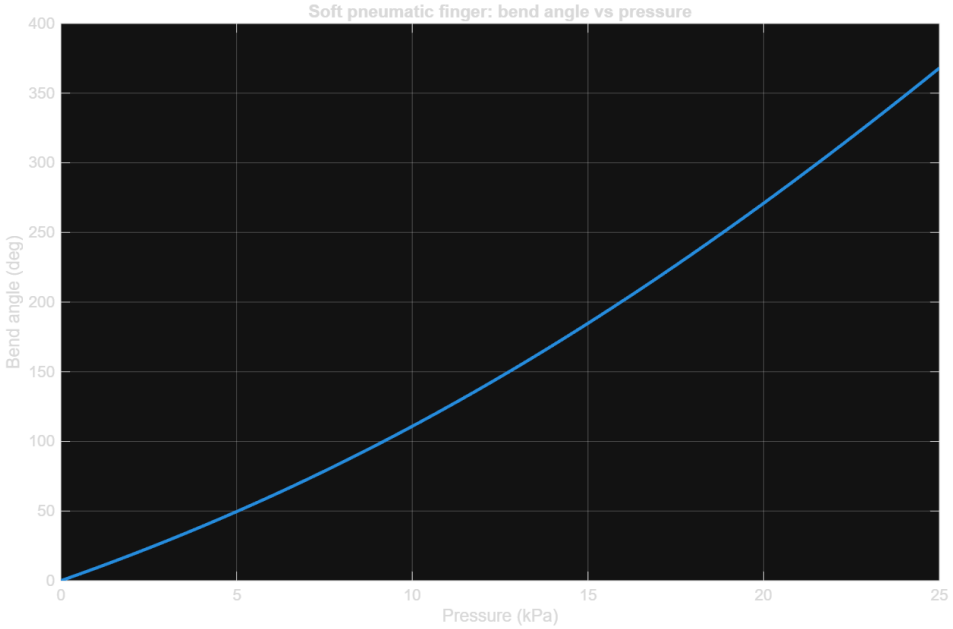


Figure 2. Bend angle vs pressure for the baseline Ecoflex 00-30 finger.

What it means: about 10 kPa (one tenth of atmospheric pressure, roughly a gentle breath) gives a right-angle-plus bend. Above about 15 kPa the predictions get unrealistic (a 368° curl). That's because the simple model ignores rubber stiffening at large stretch, the chamber ballooning sideways, and the inner wall pressing into itself. Trust it as a guide below about 15 kPa.

5. Step 3 – Which design choices matter

File: step3_param_sweep.m

Using the Step 2 model, we changed one design choice at a time and recorded the pressure needed to bend the finger 90°. This is a **parameter sweep**, a cheap way to find which knobs matter before making any moulds.

- **Wall thickness**, swept 1–4 mm. Thicker walls carry more stress, so you need more pressure.
- **Bottom layer thickness**, swept 1–5 mm. This changes the lever arm between the air and the bottom.
- **Material**: the four silicones from Step 1.

Material	Ecoflex 00-30	Ecoflex 00-50	Dragon Skin 10	Elastosil M4601
Pressure for 90°	9.1 kPa	17.1 kPa	29.2 kPa	88.6 kPa

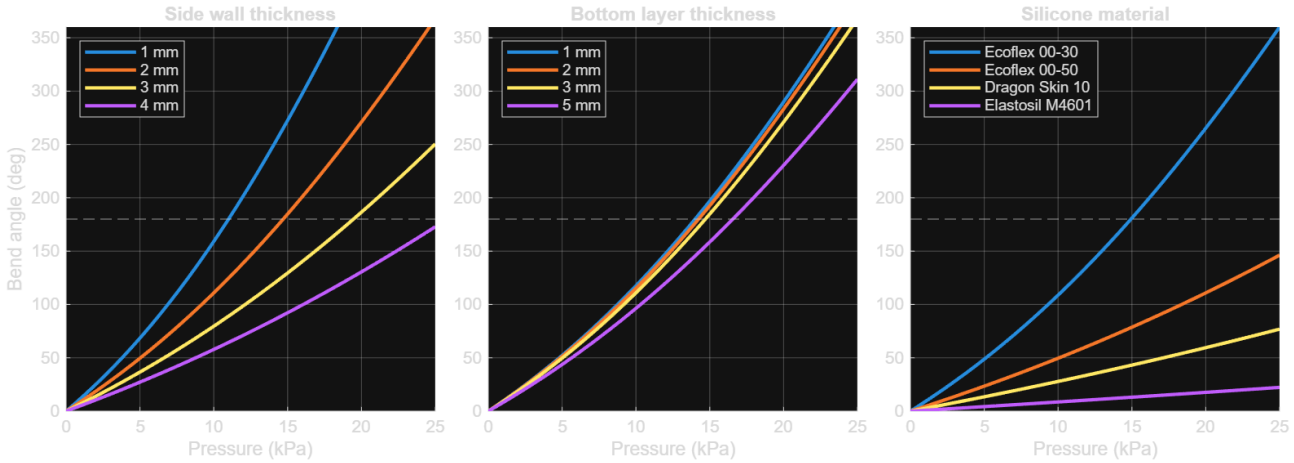


Figure 3. Pressure needed for 90° vs wall thickness, bottom-layer thickness, and material.

What it means: material choice is the biggest knob. Going from Ecoflex 00-30 to Elastosil needs almost 10× the pressure. Softer is easier to bend but weaker at gripping (see Step 6), so the choice is a trade-off.

6. Step 4 – How fast air fills and empties it

File: step4_pressure_dynamics.m

Steps 2–3 assume the pressure is already there. In reality air has to flow in through a valve, which takes time. This step simulates that over time.

The pieces of the model

- **Valve flow:** air passes through a small hole (orifice). We used the standard compressible-flow formula, which includes "choked" flow: when the pressure difference is large, flow hits a speed limit.
- **Chamber:** treated as isothermal (constant temperature). As air enters, the pressure rises, but the rubber also expands, so the chamber acts like a spring tank. We modelled this compliance as $C = V_0 / 25\ \text{kPa}$.
- **Finger motion:** the finger doesn't jump instantly to its Step 2 angle. It has mass and damping, so we modelled it like a soft spring (natural frequency 4 Hz, damping ratio 0.15, so lightly bouncy).
- **Solver:** `ode15s`, a MATLAB solver suited to systems with fast and slow parts mixed together.

Test: 10 kPa supply. Open the inlet from 0–1 s, hold, then vent from 2–3 s. We tried three valve hole sizes.

Valve hole	Time to 90% of final bend	Result
0.3 mm	never reaches it	only 47° in 1 s; too slow
0.6 mm	0.638 s	111°, smooth
1.2 mm	0.16 s	peaks at 130° (overshoot), settles at 112°

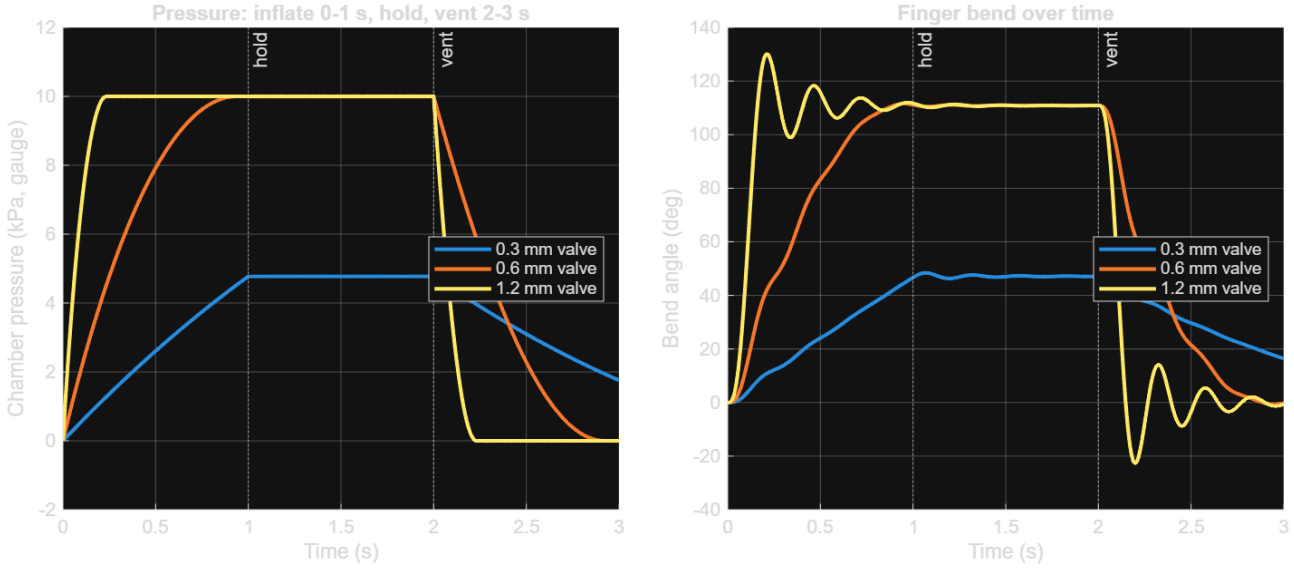


Figure 4. Chamber pressure and bend angle over time for three valve sizes.

What it means: valve size sets the speed. A bigger hole is fast but causes overshoot (the finger swings past its target). On venting the model swings to about –22°, which a real finger wouldn't do. It's a limitation of the simple spring model, not real behaviour.

Coding note: a variable was first named `inf`, which clashes with MATLAB's built-in infinity, so we renamed it `inflate`.

7. Step 5 – Holding an exact angle (control)

File: step5_pid_control.m

A gripper needs to reach and hold a chosen angle, even if the rubber isn't exactly what we think. To test this honestly, the simulated "real" finger was made **stiffer than the model believes** (μ = 30 kPa vs 25 kPa), like a real cast that came out slightly different. Supply was 20 kPa with 1.2 mm valves. The target angle stepped 0° → 60° → 100° → 30°.

First attempt: one controller (failed)

A single PID controller reading the angle and driving the valve directly overshot by 89°. Air pressure builds up with a delay, so by the time the angle is right, too much air is already in.

Second attempt: two controllers stacked (cascade)

- **Outer loop (angle → pressure):** looks at the angle error and decides what pressure the finger should have. It starts from the Step 2 model's guess (*feedforward*) and then PID corrects for the model being wrong. Gains: K_p = 0.02 kPa/deg, K_i = 0.1, K_d = 0.002, derivative filter 0.02 s.
- **Inner loop (pressure → valve):** a fast proportional controller that opens the inlet or vent valve (as a PWM duty cycle) to hit that pressure. K_p = 0.10 per kPa.
- **Realism added:** runs at 200 Hz like a microcontroller, includes sensor noise, and has *anti-windup*, which stops the integral term building up while the valve is already fully open.
- **Tuning:** a grid search tried many gain combinations and kept the best.

Target	60°	100°	30°
Pressure-only (no angle feedback): final error	–11°	–20°	–5°
Cascade: final error	–1.2°	–0.7°	+0.2°
Cascade: overshoot	2.7°	0.9°	1.6°
Cascade: settling time	1.27 s	1.01 s	0.82 s

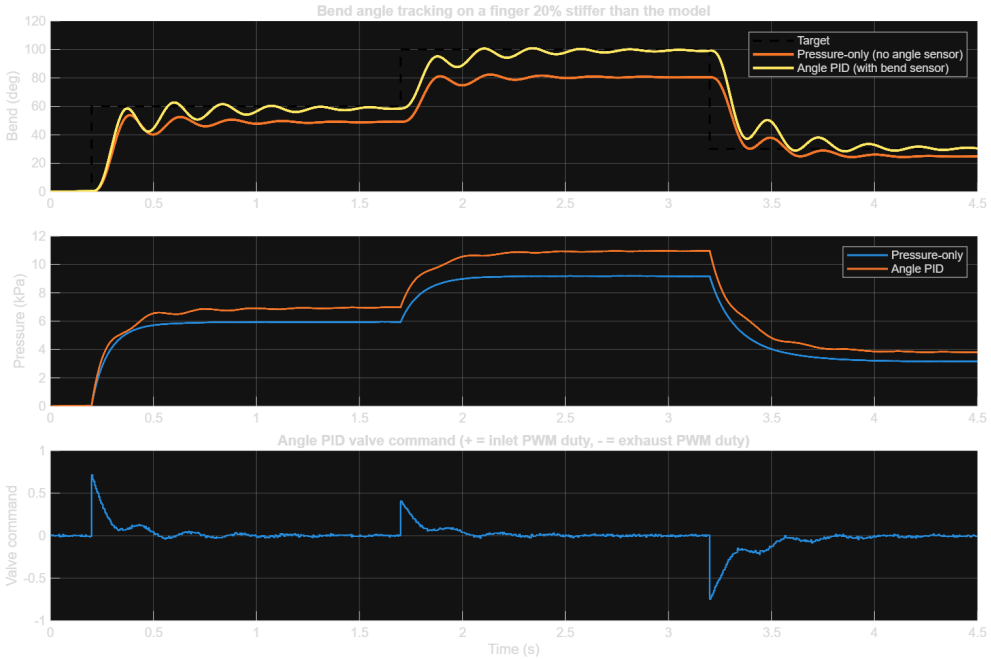


Figure 5. Cascade control tracking 60°, 100° and 30° on a finger stiffer than the model.

What it means: just setting a pressure from the model gives errors up to 20°, because the model is never exact. Measuring the angle and correcting brings errors to about 1°. This is the controller to build on an ESP32 in the lab. It needs a bend sensor (e.g. a flex sensor or IMU) plus a pressure sensor.

8. Step 6 – How hard it can squeeze

File: step6_grip_force.m

Imagine the finger bends until it touches an object at angle θ_c, then you keep adding air. The finger can't bend further, so the extra turning effect pushes on the object. Tip force = (air moment – moment the rubber uses up being bent to θ_c) ÷ length:

$$F_{tip} = (P \cdot A_c \cdot y_c - M_{rubber}(\kappa_c)) / L$$

Test case: holding an egg

- Egg mass 60 g, three fingers, friction coefficient 0.5, safety factor 2
- Force needed per finger = 0.78 N
- At 20 kPa the Ecoflex finger (contact at 30°) only gives about 0.29 N, not enough. So we extended the pressure range to 100 kPa.

Material	Contact at 30°	Contact at 60°
Ecoflex 00-30	49 kPa	51.5 kPa
Dragon Skin 10	56.5 kPa	66 kPa

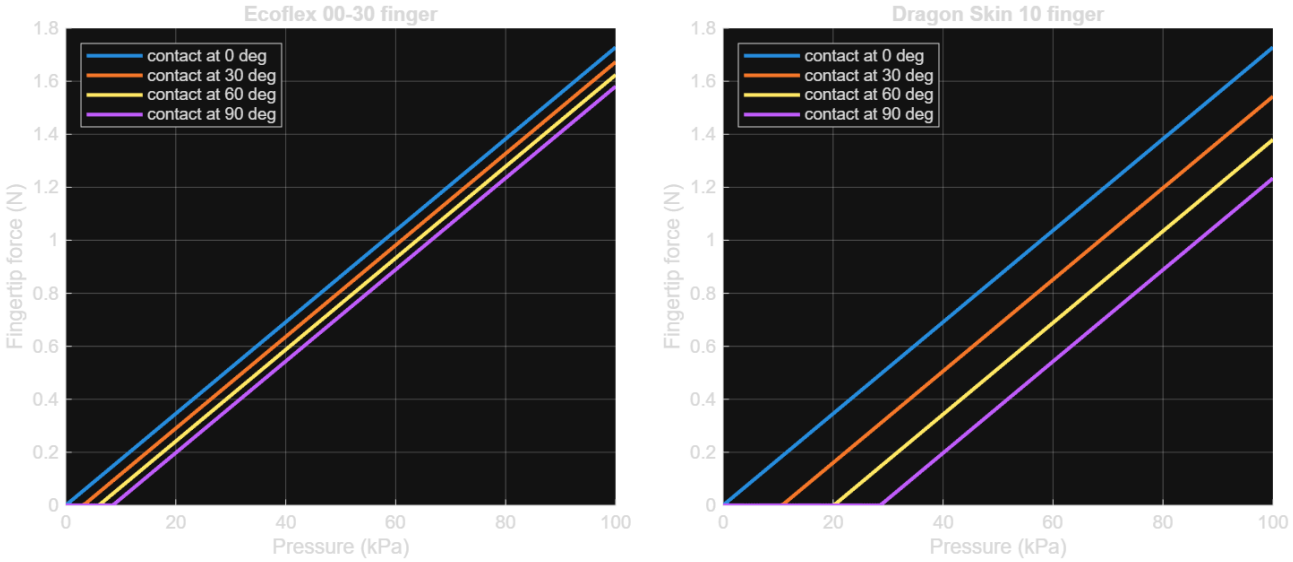


Figure 6. Tip force vs pressure, and the egg-holding threshold of 0.78 N.

What it means: a fingertip pinch needs about 50 kPa, which is well beyond where the Step 2 model is accurate, so these are rough numbers. In practice, soft grippers wrap around (cup) objects and touch along their whole length, which needs much less force than a fingertip pinch.

9. Step 7 – Getting SoRoSim working

File: step7_sorosim_demo.m

Our Steps 2–6 model is hand-built and simplified. SoRoSim is a research tool that models the finger as a **Cosserat rod**: a flexible rod whose every point can bend, twist, stretch and shear, solved with the *Geometric Variable Strain* method. First we checked that it ran correctly using its built-in example.

- Example: *SoftManipulator*, a 0.9 m soft rod, Young's modulus 1 MPa, with 6 cables
- We pulled cable 1 from 0 to 4 N and recorded the bend
- This is where we discovered the MEX mismatch (Section 2.3) and added the shims

Result: 14.47° of bend per newton, and perfectly linear, as expected for small strains. SoRoSim worked.

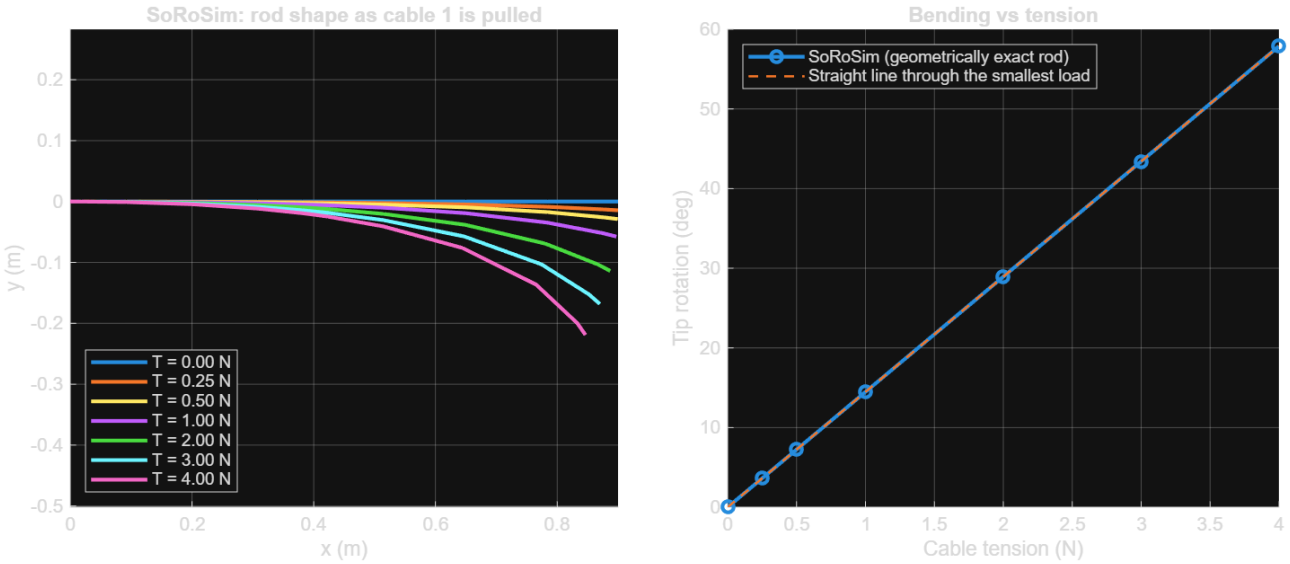


Figure 7. SoRoSim built-in soft manipulator bending under cable tension.

Warning: step7_sorosim_demo.m starts with `clear`, which wipes your workspace. Save `L` and `F` to a .mat file before running it.

10. Step 7b – Our finger in SoRoSim

Files: finger_sorosim.mat (built by you in the GUI), step7b_sorosim_finger.m

10.1 The trick: using a cable to imitate air

SoRoSim has no built-in "air chamber" option, but it does have cables. A cable running along the finger, offset from its centre, also creates a bending moment. If we pick the cable tension so that its moment equals the air's moment, the finger bends the same way:

$$\mathbf{T \times 0.008\ m = P \cdot A_c \cdot y_c \ \Rightarrow \ T = P \cdot A_c \cdot y_c / 0.008}$$

The cable sits 8 mm from the rod's centre line (z = –8 mm), on the side of the stiff bottom layer.

10.2 Building the finger in the SoRoSim GUI

A. Create the link (`L = SorosimLink`). Values entered in the dialogs:

Setting	Value	Why
Joint	N (none)	Finger base is fixed, with no rigid joint
Cross-section	Rectangular	The finger is a rectangular bar
Joint stiffness / divisions	0 / 1	No joint; one soft section
Density	1070 kg/m³	Ecoflex density
Young's modulus E	76 000 Pa	E = 3μ for rubber (3 × 25.4 kPa)
Poisson's ratio	0.5	Rubber keeps its volume
Damping	1×10^{4}	Helps the solver settle
Length, height, width	0.1 m, 0.017 m, 0.02 m (constant)	Same as the baseline finger

B. Create the linkage (`F = SorosimLinkage(L)`):

- 1 link, starting at the origin
- Strain basis: **bending about y, order 2** (the bend can vary along the length) plus **elongation along x, order 0** (uniform stretch)
- 10 Gauss points (where SoRoSim evaluates the rod along its length)
- 0 closed chains, gravity off, 0 point wrenches
- Actuation: yes, 1 cable. The broken cable dialog was skipped by the CableActuation shim (Section 2.3)
- Result: **4 degrees of freedom, 1 actuator**
- Saved with `save('finger_sorosim.mat','L','F')`

10.3 Running the comparison

step7b_sorosim_finger.m is written as a **function**, so it runs in its own space and can't overwrite your workspace variables. It loads finger_sorosim.mat, converts each pressure to a cable tension, asks SoRoSim for the static shape, measures the tip angle, and plots it next to the Step 2 result.

Pressure (kPa)	1	5	10	12	15
SoRoSim bend	11.5°	57.5°	114.9°	137.9°	172.4°
Step 2 model bend	8.9°	48.7°	108.7°	136.2°	181.0°

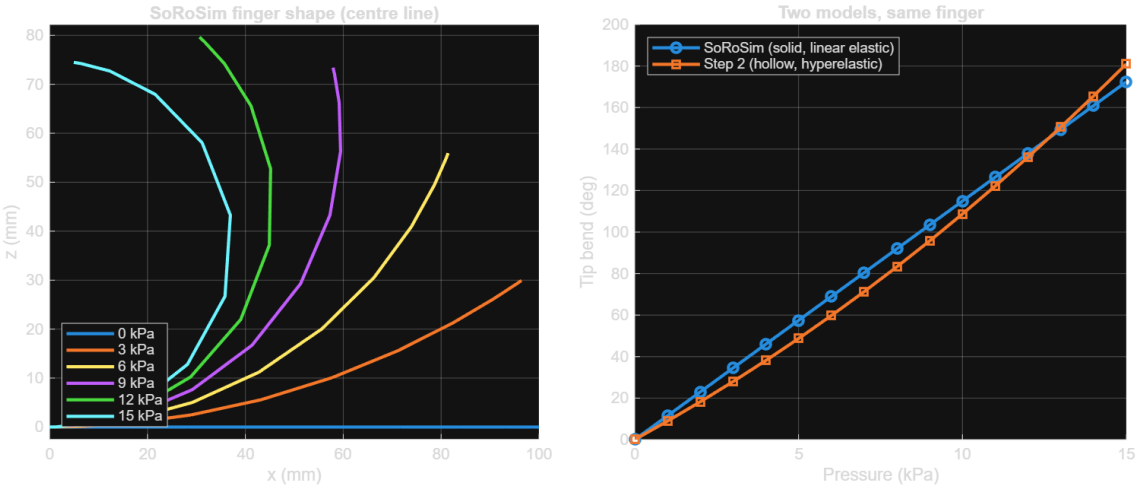


Figure 8. SoRoSim (Cosserat rod) vs our Step 2 model for the same finger.

What it means: two very different methods agree within about 10° between 5 and 15 kPa. SoRoSim's line is straight (linear material), while our model curves upward because neo-Hookean rubber softens a little at moderate stretch. That agreement gives some confidence in the Step 2 model at low pressure. But both are models, and only lab measurements can say which is closer to reality.

11. Problems we hit and how we fixed them

Problem	Cause	Fix
MCP "Server disconnected" / "Failed to create file"	MATLAB path typed into Log Folder; default temp log folder issue	Proper install path, project logs folder, session mode new
SoRoSim MEX functions wouldn't run	Compiled with a different MATLAB Coder version	sorosim_shims: same-named .m wrappers that call the source code
Rigid-joint functions crashed on empty inputs	Empty Phi/q had the wrong size	reshape() inside the shims
Cable dialog crash at CableActuation line 33	GUI saved an empty div_endi	CableActuation.m shim creates the cable directly
"Input must be of SorosimLink class"	Claude overwrote your variable L in the shared MATLAB session	You rebuilt L. Claude now uses zz_ temp names and function files
Single-loop PID overshot 89°	Pressure lag	Cascade controller (Step 5)
Couldn't hold the egg at 40 kPa	Range too low	Extended the sweep to 100 kPa
exportgraphics "Could not open file"	PNG locked (likely OneDrive syncing or an open viewer)	try/catch saves to a timestamped filename instead

Tip: if file locks keep happening, move the project out of the OneDrive folder (for example to C:\SoftFinger).

12. What the model can't tell us yet

- Bending above about 15 kPa is over-predicted (no rubber stiffening, no sideways ballooning, no wall contact).
- The chamber is treated as one smooth channel, while real PneuNets have separate ribbed chambers.
- The vent undershoot in Step 4 (–22°) is not physical.
- Grip force (Step 6) is a fingertip-pinch estimate only.
- The SoRoSim cable is an equivalent-moment stand-in for air pressure, not real pressure loading.
- No material values were measured. All come from the literature.

Measurement plan (December, in the lab)

- **Bend vs pressure:** photograph the finger at 0–20 kPa in 2 kPa steps and measure the angles. Compare with Figures 2 and 8.
- **Speed:** film inflate and vent with a phone (slow-motion) and compare with Step 4 to tune the valve and damping values.
- **Tip force:** press the tip on a load cell or kitchen scale at several pressures and compare with Step 6.
- **Control:** run the Step 5 cascade controller on an ESP32 with a pressure sensor and a bend sensor.

Next modelling steps

- Swap neo-Hookean for Yeoh in Step 2 and refit C_{10}/C_{20} to the lab data
- Add chamber ballooning and ribbed-chamber geometry
- Fit the Step 4 dynamics to the inflation video
- Simulate a 3-finger gripper in SoRoSim with contact

13. File list

File	What it is
step1_material_model.m / .png	Neo-Hookean vs Yeoh rubber models, four materials
step2_pressure_bend.m / step2_bend_vs_pressure.png	Moment balance, pressure to bend angle
step3_param_sweep.m / .png	Wall, bottom layer and material sweep
step4_pressure_dynamics.m / .png	Valve flow plus chamber plus finger motion over time
step5_pid_control.m / .png	Cascade PID with model mismatch and noise
step6_grip_force.m / .png	Tip force and the egg test
step7_sorosim_demo.m / .png	SoRoSim built-in manipulator check (starts with clear)
step7b_sorosim_finger.m / .png	Our finger in SoRoSim vs Step 2 (function, workspace-safe)
finger_sorosim.mat	Your GUI-built SoRoSim link L and linkage F
sorosim_shims/	MEX wrappers and the CableActuation fix (must be first on the path)
sorosim_runs/, logs/	SoRoSim outputs, MCP server logs
README.md	Short summary of all steps and setup notes